RESEARCH-ARTICLE

Prompting for Comprehension: Exploring the Intersection of Explain in Plain English Questions and Prompt Writing

DAVID H. SMITH, University of Illinois Urbana-Champaign, Urbana, IL, United States

PAUL DENNY, University of Auckland, Auckland, AUK, New Zealand

MAX FOWLER, University of Illinois Urbana-Champaign, Urbana, IL, United States

Open Access Support provided by:

University of Auckland

University of Illinois Urbana-Champaign

Published: 15 July 2024

L@S '24: Eleventh ACM Conference on Learning @ Scale
July 18 - 20, 2024
GA, Atlanta, USA

https://doi.org/10.1145/3657604.3662039
ISBN: 9798400706332

Prompting for Comprehension: Exploring the Intersection of Explain in Plain English Questions and Prompt Writing

David H. Smith IV
University of Illinois
Urbana, IL, USA
dhsmith2@illinois.edu

Paul Denny
University of Auckland
Auckland, New Zealand
paul@cs.auckland.ac.nz

Max Fowler
University of Illinois
Urbana, IL, USA
mfowler5@illinois.edu

ABSTRACT

Learning to program requires the development of a variety of skills including the ability to read, comprehend, and communicate the purpose of code. In the age of large language models (LLMs), where code can be generated automatically, developing these skills is more important than ever for novice programmers. The ability to write precise natural language descriptions of desired behavior is essential for eliciting code from an LLM, and the code that is generated must be understood in order to evaluate its correctness and suitability. In introductory computer science courses, a common question type used to develop and assess code comprehension skill is the 'Explain in Plain English' (EiPE) question. In these questions, students are shown a segment of code and asked to provide a natural language description of that code's purpose. The adoption of EiPE questions at scale has been hindered by: 1) the difficulty of automatically grading short answer responses and 2) the ability to provide effective and transparent feedback to students. To address these shortcomings, we explore and evaluate a grading approach where a student's EiPE response is used to generate code via an LLM, and that code is evaluated against test cases to determine if the description of the code was accurate. This provides a scalable approach to creating code comprehension questions and enables feedback both through the code generated from a student's description and the results of test cases run on that code. We evaluate students' success in completing these tasks, their use of the feedback provided by the system, and their perceptions of the activity.

CCS CONCEPTS

• **Social and professional topics → Computing education**.

KEYWORDS

Explain in plain English, EiPE, Large language models, LLMs, Code comprehension, Prompting, Introductory programming, CS1

ACM Reference Format:
David H. Smith IV, Paul Denny, and Max Fowler. 2024. Prompting for Comprehension: Exploring the Intersection of Explain in Plain English Questions and Prompt Writing. In *Proceedings of the Eleventh ACM Conference on Learning @ Scale (L@S '24), July 18–20, 2024, Atlanta, GA, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3657604.3662039

L@S '24, July 18–20, 2024, Atlanta, GA, USA

ACM ISBN 979-8-4007-0633-2/24/07
https://doi.org/10.1145/3657604.3662039

1 INTRODUCTION

Developing the ability to program is a complex task which is often considered to be composed of a variety of skills [35, 37]. Though these skills are highly inter-related, they are often addressed pedagogically as being distinct [21, 50]. Among these skills is the ability to comprehend and communicate the purpose of code [36]. A common approach to teaching and assessing this skill involves the use of "Explain in Plain English" (EiPE) questions. EiPE questions present students with a segment of code and ask them to explain what the code does using natural language [38]. Correctly answering such questions requires students to both comprehend the code segment and be able to describe the code's purpose.

Rubrics for grading EiPE questions are often informed by the "Structure of the Observed Learning Outcome" (SOLO) taxonomy [3], specifically as it was adapated by Lister et al. [36] for use in the context of code comprehension. In the context of grading, this uses the first four levels from the SOLO taxonomy: Prestructural, Unistructural, Multistructural, and Relational. The first two levels are used to categorize student responses that demonstrate either no understanding or some fundamental misconceptions of the constructs used in the code, respectively. The third level indicates that a student understands all aspects of the code, but fails to connect them into a high-level purpose. Finally, relational responses are those that describe the code's purpose without attending to its implementation. Instructors using EiPE questions often prize responses that lean towards relational, or what some call "high-level", descriptions of code and value the questions for their ability to develop and assess this level of comprehension [20].

Though the development of code comprehension skills has long been considered a key learning objective in CS1 courses, the emergence of code generation tools has amplified its importance. Many researchers and commentators have suggested that tools that support the development of "prompting skills" should be integrated into CS1 courses [12, 13, 43, 46]. Integral to the use of these tools is 1) the ability to craft effective prompts and 2) the ability to evaluate the code generated by the prompts. Each of these tasks requires the ability to comprehend code and the former bears a striking similarity to the task students are given in EiPE questions. As such, research surrounding the use of EiPE questions and code comprehension more broadly may provide insight into the development of tools that support students when learning to program with the presence of code generation tools.

Though EiPE questions may share a touchpoint with the task of crafting prompts the overhead of grading short answer responses limits their use at scale particularly in formative contexts. While autograders have been developed for EiPE questions, they require a

large corpus of human-labeled data to train a model for each question and provide only dichotomous feedback [1, 2, 6, 19]. This limits the actionability of the feedback, the scalability of the approach, and the transparency of the grading process from the student's perspective [26]. The issue of transparency is of particular concern as it is a key factor in determining students' trust in automatically graded results and therefore their ability to properly evaluate their own performance [34].

To address these shortcomings, in this paper we explore a novel approach to teaching and assessing code comprehension inspired by EiPE questions. In this approach, students are shown a segment of code and asked to write a prompt that would generate functionally equivalent code. This prompt is then used to generate code using an LLM (OpenAI's GPT-3.5), and that code is evaluated using instructor-defined unit tests. Students are then provided with both the generated code and the results of the tests run on that code as feedback. This addresses issues related to the scalability of grading EiPE questions, as the instructor overhead of authoring new questions is reduced to that of a typical automatically graded code-writing question. Additionally, the feedback provided to students has the potential to be more actionable and transparent than previously proposed auto-grading approaches. In evaluating this exercise as a means of teaching and assessing code comprehension, we seek to answer the following research questions:

RQ1: How successful are students at crafting code explanation prompts that induce an LLM to generate functionally equivalent code?

RQ2: What is the relationship, if any, between the success of a student's prompt and the classification of that prompt with respect to the categories of the SOLO taxonomy?

RQ3: What are students' thoughts about the code explanation activity in comparison to more traditional code writing tasks, and do they see it as an accurate way to assess their comprehension of code?

RQ4: What are students' perceptions of the feedback (i.e., unit-tests, generated code) and its ability to help them improve their prompts?

RQ5: What are students' general perceptions on LLM code generation and its integration into computing courses?

The goal of **RQ1** and **RQ2** is to evaluate the effectiveness of this approach as a means of assessing and teaching code comprehension. **RQ3** and **RQ4** seek to understand students' perceptions of the activity and the value they see in the feedback provided by the autograder for improving their prompt crafting skill. Finally, **RQ5** seeks to understand students' perceptions of the use of LLMs in introductory CS education more generally towards informing the integration of this activity and activities like it into the broader CS curriculum.

2 BACKGROUND AND RELATED WORK

2.1 Programming Skills and Code Comprehension

What skills does it take to program? Addressing this question involves considering how humans, particularly experts, interact with code. At a basic level programmers require the ability to understand the syntax and semantics of the language(s) they are using [48]. Developing this understanding is often addressed at the novice level through the use of syntax drills which familiarize students with a language's constructs [14, 16]. Beyond this, skills such as the ability to write, trace, and read code emerge as the skills many CS1 courses focus on developing [21, 35, 37, 50]. Other skills such as debugging [9], program design [27], and code style [28, 41] exist but are less often cited as being a core focus of CS1 courses.

Code reading tasks involving explanation of code differ from other tasks as the responses they elicit are in natural language rather than code and are therefore less easy to verify for correctness. A common method of assessing code comprehension involves the use of "Explain in Plain English" (EiPE) questions which require students to consider a segment of code and understand (and explain) its purpose [38]. Though responding to these questions often involves tracing [39], the form of response is natural language. As is the case with perhaps all short answer responses, this leads to subjectivity in what constitutes a correct answer. This further distances the grading of these questions from the deterministic nature of code writing and tracing tasks.

Current grading practices for EiPE questions appear to trace their roots to the "Structure of the Observed Learning Outcome" (SOLO) taxonomy [6, 38]. The original taxonomy defined five levels of comprehension: 1) Prestructural: the student lacks understanding of any concepts relating to the task, 2) Unistructural: the student has some understanding of the task but makes an error, omission, or displays a misconception, 3) Multistructural: the student is able to describe all aspects of the task but does not integrate them, 4) Relational: the student is able to integrate all aspects of the task into a single combined description, and 5) Extended Abstract: the student is able to generalize the task to other contexts [3]. Lister et al. [36] adapted and contextualized the first four levels of the taxonomy to the context of evaluating EiPE responses finding that students who performed well in other programming tasks tended to provide more relational responses to EiPE questions. A recent study evaluating instructor grading practices for EiPE questions found that instructors value high-level descriptions of code that focus on the code's purpose rather than low-level descriptions of the code's implementation [20].

2.2 Automatic Short Answer Grading and EiPE Questions

Compared to the skills of code writing and tracing, code comprehension tasks are comparatively difficulty to grade automatically. Code writing questions can be autograded by developing unit tests which validate if a student's submission is functionally correct. Tracing questions can present students with a segment of code with a given input and ask them to provide the output. Code comprehension questions are more difficult as to demonstrate comprehension, a natural language response is often required which is less easily verified. This in turn limits the amount of practice students can receive on these questions, the timeliness of the feedback they receive, and the ability to use these questions at scale.

The issue of automatically grading short-answer questions is a long-standing problem particularly in fields where short answer

questions are more prevalent such as the natural sciences, reading comprehension, and the liberal arts [22]. To address this issue "Automatic Short Answer Grading" (ASAG) systems have been developed over the past three decades using a variety of techniques from concept mapping to more advanced machine and deep learning models [5]. However, as noted by Burrows et al. [5], these systems are difficult to readily adopt as previous models require features that characterize correct answers to be developed on a per question basis. Though more recent deep learning models alleviate this requirement they still require large amounts of human labeled data to train and models to be created on a per question basis. More recent approaches that seek to address this limitation have involved the use of large language models (LLMs) and have achieved impressive results [17, 25, 40].

Despite the difficulty of ASAG the approach has been applied to EiPE questions. Questions in the domain of programming benefit from the field having a well defined lexicon and typically a smaller set of plausible desired responses. The first known application of ASAG grading for EiPE questions was done by Azad et al. [2] who used a bigram logistic classifier trained on 500–600 human–labeled responses for each question. Followup investigations on the same autograder by Fowler et al. [19] found that the autograder achieved accuracy with ground truth that was similar to a trained teaching assistant (87–89%).

Though the results of Fowler et al. [19] are highly promising, their system retains the one model per question limitation of other ASAG approaches. Additionally, the system provides only dichotomous feedback (correct/incorrect) which limits the ability of students to understand why their responses may have failed and how they might improve. This, in turn, limits the transparency of the grading process which can lead students to distrust the correctness of the feedback they receive [26, 34]. In the context of programming assignments, providing only dichotomous feedback in lieu of more detailed feedback (i.e., hints, tests cases) has been shown to lead to students engaging in less efficient trial-and-error problem solving [23, 24]. These limitations motivate the need for other approaches to grading EiPE questions that achieve the same goals of providing scalable code comprehension tasks while improving the transparency of the grading process and the ease with which new questions can be authored.

2.3 Large Language Models in Introductory CS Education

Large language models, such as OpenAI's GPT-3.5 and GPT-4, have already shown to be highly capable of generating solutions to a variety of question types present in introductory CS courses [7, 15, 18, 31, 44]. This has caused concern among many computing educators who worry that the over-use of LLMs may hamper student learning [8, 42]. Others have suggested their use should be integrated into classrooms to support student learning likening it to the advent and integration of handheld calculators into math education [10, 30, 42]. Given the already widespread adoption of LLM powered tools for supporting development (GitHub Copilot, Amazon Code Wisperer) and the emergence of open source LLMs tuned for code generation (e.g., StarCoder [33], Code Llama [45]) there is a growing need for research investigating how these tools should be integrated into current approaches for teaching introductory computer science.

Central to teaching with LLMs is the need to teach students to successfully prompt language models for code generation and to understand how students are using LLMs [29]. With respect to the former, investigations by Denny et al. [12] showed that, though GPT-3.5 was able to generate correct code for the majority of questions in a corpus of Python CS1 programming questions, small refinements to the prompts further improved the correctness of the generated code. This finding highlights the need for prompting to be taught as a skill in order to improve students' abilities to use LLMs for code generation. Denny et al. [13] introduced "Prompt Problems", an approach to teaching students to prompt LLMs that provides students with a visual representation of input and output, asks the student to infer the operation(s) being performed, generates code based on a description they provide, and uses unit tests to determine if the generated code is correct [43]. Similarly, Smith IV and Zilles [46, 47] developed and evaluated an approach to grading EiPE questions that generates code from student responses and uses unit tests to determine if that code is correct as a proxy for the correctness of the student's description. They found the approach achieved moderate agreement with human graders and note that future work should evaluate student perceptions of this grading approach as well as the role of the feedback in supporting student learning. Recent work applied the method outlined by Smith IV and Zilles to generate immediate feedback for students solving EiPE questions, finding that relational (or high-level) prompts tended to be more successful [11]. In the current work, we provide a more detailed analysis of this relationship, and investigate how students make use of the feedback available to them. We also explore several properties of student prompts including both correctness and consistency, and seek students' thoughts on the integration of LLM-based activities into their courses. Supporting students through the use of LLMs and developing their skills when using them is a nascent yet timely field that is both ripe for exploration and likely to grow in importance as LLMs continue to be integrated into development tools and CS classrooms.

3 METHODOLOGY

Table 1: Questions for Lab A (Loops, Arrays and Functions) and Lab B (Strings, Text Processing and 2D arrays)

Lab	Question	Description
A	Q1	Sum between a and b inclusive
A	Q2	Count even numbers in array
A	Q3	Index of last zero (see Figure 1)
A	Q4	Sum positive values
B	Q1	Reverse a string
B	Q2	Calculate sum of row in 2D array
B	Q3	Is a vowel contained in a string?
B	Q4	Does a string contain a substring?

This study was conducted in a large introductory programming course at the University of Auckland, a large research university in New Zealand. This course is required for all students in the Faculty of Engineering and is taken by students in their first year of study. The course is taught in C and MATLAB and covers standard

CS1 material in the context of both languages. At the time of the study 889 students were enrolled in the course of which 841 participated. Access to the data involved in this study was approved by the University of Auckland Human Participants Ethics Committee (UAHPEC) under protocol UAHPEC25279.

3.1 Exercise Design

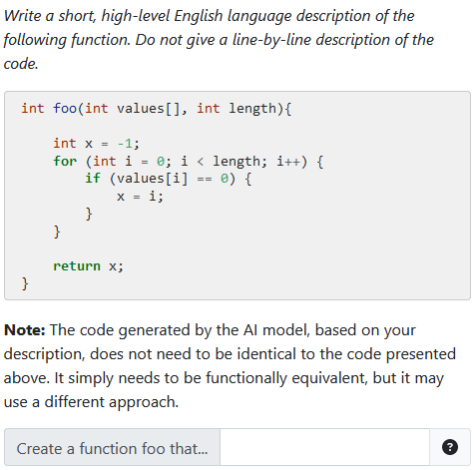


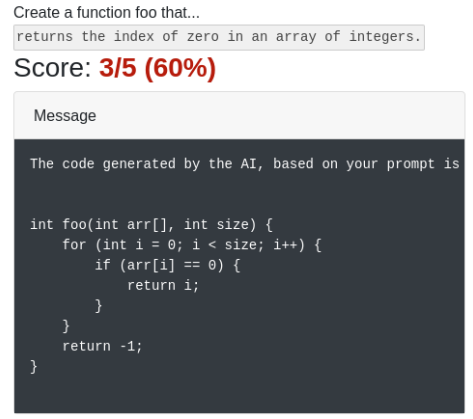
Figure 1: Example of a EiPE problem displaying code that returns the index of the last zero in an array. The code that is shown has the ability to highlight and copy it disabled to dissuade students from copying it into ChatGPT.

In the tradition of EiPE questions, for each task students were presented with a single function and a prompt indicating that they should describe the function in plain English (Figure 1). To prevent giving away the code's purpose, the variables were largely replaced with generic names and each function was named `foo`. Questions were developed and given to students via PrairieLearn, an open-source online assessment platform [49]. After submitting a prompt for a given question, students are shown the code that was generated by GPT-3.5 (Figure 2a) and the results of all test cases run on the generated code (Figure 2b).

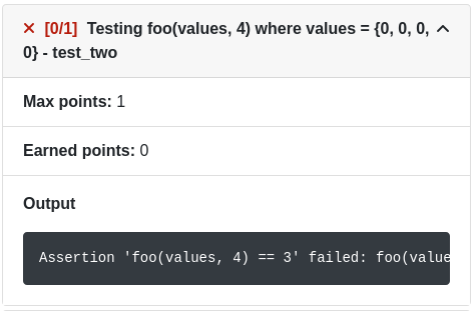
A total of eight questions were used across two separate lab assignments, with four questions being used in each lab. These labs occurred in the 8th and 10th weeks of the 12 week semester and we refer to these labs as Lab A and Lab B, respectively. Lab A focused on loops and 1-dimensional arrays of integers whereas Lab B contained more advanced questions involving strings and 2-dimensional arrays (Table 1). Students were allowed up to 20 attempts per question but were not penalized for incorrect submissions. Other than a 255-character limit imposed on Lab B to dissuade students from using excessively long prompts, the question format remained identical across the two labs.

3.2 Analysis of Student's Prompts Success

To address **RQ1** we classify the success of a prompt as simply whether or not the code generated was functionally equivalent to the code which the student was being asked to describe. In addressing this research question we look at 1) the average number of attempts students made prior to successfully completing the task, 2) the standard deviation of the number of attempts, and 3) the percentage of students who successfully completed each task. This is done to understand both the overall difficulty of the tasks and the difficulty of the tasks relative to each other. Additionally, given students had limited exposure to activities of this type prior to the lab activity these results also provide insight into the learning curve associated with this type of activity for students who are otherwise uninitiated.



(a) Students are shown the code that was generated by GPT-3.5 from their prompt.



(b) Students are also shown the results of test cases run on the generated code along with the function call, input data, and the assertion that failed.

Figure 2: The two modes of feedback provided to students in the event of an incorrect response.

3.3 Analysis of Student's Prompts with Respect to the SOLO Taxonomy

To address **RQ2** we classify the prompts with respect to the SOLO taxonomy as contextualized by Lister et al. [36] for the domain of programming. We use the following categories and definitions:

- **Prestructural**: The student gives a prompt which shows a significant lack of understanding of the code.

Table 2: Reflection questions, organised by research question, shown to students after attempting the code explanation tasks.

RQ	Type	Question
RQ3	Open	Please reflect on the code comprehension task and comment on how you found this type of an exercise compared to a typical programming task.
RQ3	Likert	Having AI language models generate code from natural language descriptions is an accurate way to evaluate code comprehension skills
RQ3	Open	Do you have any comments about this lab?
RQ4	Open	Please reflect on your use of the system's feedback (e.g., test-cases, generated code).
RQ5	Open	Please comment on your thoughts regarding the capability of large language models for generating code
RQ5	Likert	Students should be taught how to use Generative AI tools effectively for their future careers.

- **Unistructural**: The student gives a prompt which contains errors or misconceptions but still shows some understanding of the code and its structures.
- **Multistructural**: The student gives a prompt which contains multiple correct ideas but does not integrate them into a single, high-level description of the code's singular purpose.
- **Relational**: The student gives a prompt that correctly describes the code's purpose at a high level without attending to details of it's implementation.

An important consideration to note is that it is possible for a question to contain both a *multistructural* description followed by a *relational* summary. Following the example of Lister et al. [36] we code those responses as being *relational*.

Prompts were classified by two researchers. First, 200 responses were randomly selected from two questions (100 from Q1 - Lab A and 100 from Q2 - Lab B) and deductively coded by each of the researchers independently using the codes described above. Using these results, inter-rater reliability was calculated using Cohen's κ and found to be $\kappa = 0.79$. Following this, the researchers reconciled those codes where disagreement occurred and then each coded 100 prompts for each of the remaining six questions.

We use the data resulting from this process to explore the relationship between success of a prompt and the prompt's SOLO classification. Here we operationalize success in two ways: (1) *correctness:* whether or not the code generated was functionally equivalent to the code the student was being asked to describe and (2) *consistency:* given LLMs are non deterministic, we examine how consistent a prompt is in generating correct or incorrect code.

Analysis of Correctness: For correctness, we graph the proportion of prompts belonging to each category which were graded as correct. To further address whether *relational* prompts are more successful than *multistructural* prompts, we fit an Ordinary Least Squares (OLS) model predicting the *correctness* of the code generated by the LLM during the lab activity.

$$C = \beta_0 + \beta_1 \text{Relational} + \beta_2 \text{QID}_i + \beta_3 (\text{QID}_i \times \text{Relational}) + \epsilon$$

Here, *Relational* is a binary variable representing whether a response is *multistructural* (0) or *relational* (1) where *multistructural* is selected as the reference category. QID_i is a dummy variable representing the question number and $\text{QID}_i \times \text{SOLO}$ is the interaction term between the two. We include the interaction term to identify how the correctness of *multistructural* and *relational* prompts varies on a per question basis.

Analysis of Consistency: For consistency, prompts collected from the activity were used to generate 10 responses each at the same GPT-3.5 temperature used during the lab activity (0.7). These responses were then graded by the autograder used for their respective exercises. Using these 10 responses, we calculated the variance of correctness in order to assign a consistency score to each prompt. These results are then plotted for each question with respect to each of the SOLO categories. This is done to understand how consistently a given prompt produces a correct or incorrect solution and how this varies across categories and questions.

3.4 Analysis of Student's Perceptions

To address **RQ3-5** students were given a series of questions at the end of each lab. These questions consisted of a mix of open response and 5-point Likert scale questions. These questions and the research question they are associated with are presented in Table 2. The results to the Likert scale questions are summarized using a diverging stacked bar chart. The open-response data was analyzed using the guidelines for reflexive thematic analysis wherein codes were initially given to each response and then grouped into themes [4]. The most prevelant of these themes are presented with a description of the broader findings along with supporting quotes from the students (Section 4.3- 4.5).

4 RESULTS

4.1 RQ1: Student success at crafting code explanation prompts

The results across both Labs A and B indicate that students were extremely successful in completing the activities (Table 3). The vast majority of students who participated were able to complete the activities in both labs with the lowest completion rate being 96.3% in Lab B. Additionally, the majority of students were able to complete each exercise in 1–2 attempts even on Lab B where the exercises involved more complex code. These results are heartening as despite the novelty of the tasks students were largely able to complete the majority of these activities with apparent ease.

The final exercise in Lab B is notable not only for having the lowest completion rate (96.3%) but also for having the highest average number of attempts (2.58) and a large standard deviation in the

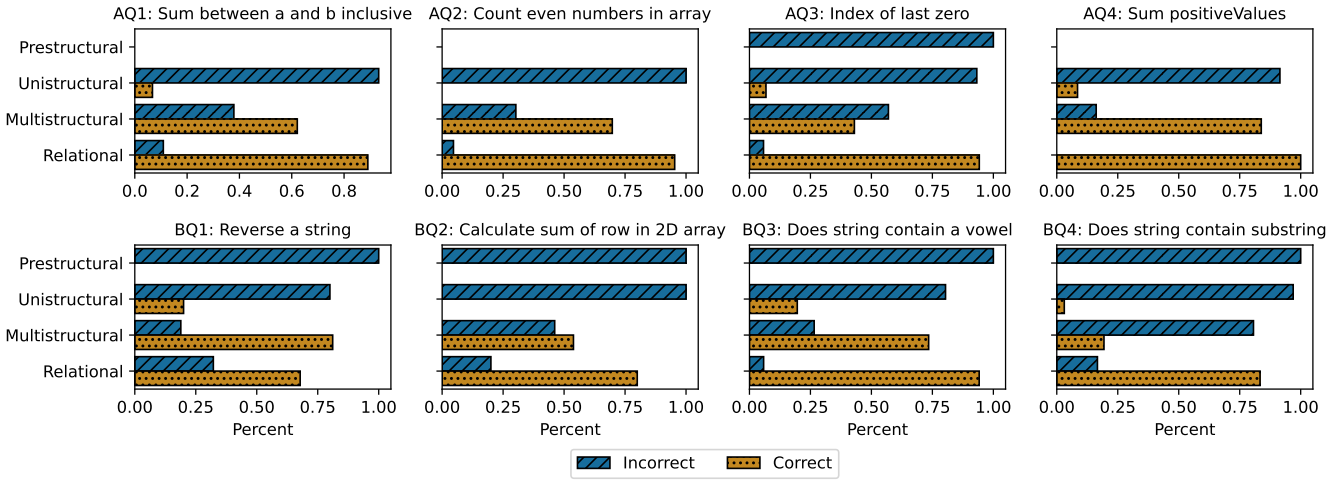


Figure 3: The proportions of prompts graded as correct vs incorrect by the autograder plotted with respect to their ranking in the SOLO taxonomy.

Table 3: The average number of submission attempts (μ), standard deviation of attempts (σ), and percentage of students who successfully completed each task.

#	Task Description	μ	σ	% Correct
Lab A	Sum between a and b inclusive	2.04	2.53	98.2
	Count even numbers in array	1.37	1.00	99.0
	Index of last zero	2.18	3.44	99.8
	Sum positive values	1.43	0.75	99.2
Lab B	Reverse a string	1.65	1.56	99.6
	Calculate sum of row in 2D array	2.03	3.11	98.1
	Is a vowel contained in a string?	1.49	1.89	98.1
	Does a string contain a substring?	2.58	6.43	96.3

number of attempts (6.43). This is perhaps unsurprising as the final exercise was the most complex and required students to understand and describe the relationship between an inner and outer loop each of which were iterating over different strings.

4.2 RQ2: Relationship between prompt success and SOLO taxonomy

4.2.1 Results for Correctness in the Activity: Figure 3 shows the proportion of prompts which generated correct code versus those that did not, plotted with respect to their SOLO classification. Overall, we find that *prestructural* and *unistructural* prompts are nearly always graded as incorrect. This is not surprising as these prompts are often missing details, contain errors, or are otherwise incomplete. Nonetheless, these results are encouraging as they suggest this grading approach is unlikely to "autocorrect" or "autocomplete" prompts and potentially create or reinforce misconceptions.

As for *multistructural* and *relational* prompts, we find that the majority of these are graded as correct. However, it is notable that a greater proportion of *multistructural* prompts are graded as incorrect compared to those that are *relational*. The results of the

Table 4: Results of regression predicting correctness of a prompt from its Relational classification and the interaction between Relational and question ID (QID).

	Coefficient	Std. Error	t-value	p-value
Intercept	0.6212	0.048	12.895	0.000***
Relational	0.2697	0.071	3.774	0.000***
AQ2	0.0765	0.064	1.194	0.233
AQ3	-0.1910	0.064	-2.982	0.003**
AQ4	0.2172	0.062	3.492	0.000***
BQ1	0.1901	0.072	2.634	0.000***
BQ2	-0.0828	0.073	-1.140	0.254
BQ3	0.1135	0.074	1.538	0.124
BQ4	-0.4277	0.069	-6.179	0.001**
AQ2:Relational	-0.0139	0.093	-0.149	0.881
AQ3:Relational	0.2424	0.099	2.444	0.015*
AQ4:Relational	-0.1081	0.099	-1.089	0.276
BQ1:Relational	-0.4031	0.096	-4.181	0.000***
BQ2:Relational	-0.0082	0.100	-0.082	0.935
BQ3:Relational	-0.0625	0.100	-0.625	0.532
BQ4:Relational	0.3701	0.099	3.757	0.000***
R^2: 0.20	F: 20.29	N.obs: 1150	DF-res: 1134	

Significance codes: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

regression analysis predicting the correctness of a prompt based on its SOLO classification (*relational* or *multistructural*) support this observation showing that *relational* prompts are significantly more likely to be graded as correct than *multistructural* prompts (Table 4). More closely examining the interaction terms shows this result does vary across individual questions. However, the general trend indicates that prompts which contain a *relational* element are at worst equally likely to be graded as correct as *multistructural* prompts and at best significantly more likely to be graded as correct.

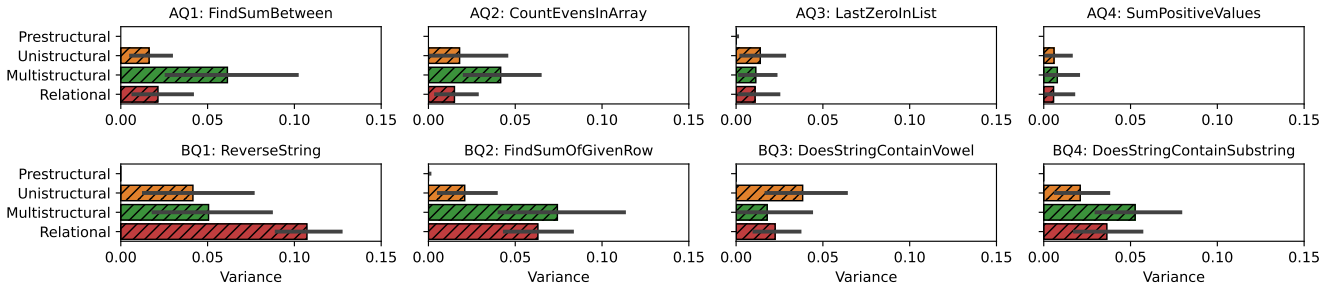


Figure 4: The average variance in the correctness of code produced by prompts belonging to each category.

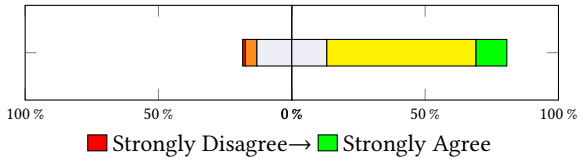


Figure 5: "Having AI language models generate code from natural language descriptions is an accurate way to evaluate code comprehension skills."

Question BQ1 (reverse a string) is the only exception to *relational* responses being graded as correct more often than *multistructural* responses. A standout feature of this question is that the code given to the students performed an in-place reversal of the string rather than returning a new string. As such, responses such as, *"write a function that reverses a string in C"* often generated code that did perform the correct operation, but did not pass the test cases as the original string was unmodified. Conversely, relational responses such as, *"takes a string as input and reverses the order of the letters and assigns the reversed string to the original string"* generated code that successfully passed the test cases. Though both of these would be considered *relational* responses, this highlights the occasional need for more specific relational prompts on the part of the students or more lenient test cases on the part of the autograder depending on the desired learning outcome.

4.2.2 Results for Consistency: Overall, the results for consistency indicate that, regardless of the SOLO category, the variance in functionality of code produced by a given prompt is relatively low (Figure 4). Across all questions, *relational* prompts exhibit a similar or lower level of variance in the results their prompts produce compared to *multistructural* prompts. Similar to the correctness results we find that the one exception to this trend is the *Reverse a string* question. Examination of the responses generated by the *relational* prompts which exhibited low consistency revealed that the prompts often missed the requirement to reverse the string in place simply stating *"reverses a string"*.

4.3 RQ3: Student perceptions of code explanation activity

Analysis of students' perceptions relating to the task itself revealed a variety of themes including novelty and engagement, the utility of teaching students to produce LLM generated code, and the potential for using prompting LLMs as a means to support the development of code comprehension. Overall, students expressed a high degree of enjoyment and engagement with the activities, particularly valuing them as a means to develop code comprehension skills.

4.3.1 Novelty and Engagement: Overall, students were extremely positive regarding their experience with the activities. Many students reported that they enjoyed the integration of AI supported activities into the course, a sentiment also reflected in the Likert scale responses (Figure 6).

> *"I actually found it a little bit entertaining since it was unbelievable to me that a code could be written directly from a sentence or two"*

> *"The task was a lot of fun to do and it was interesting to see how the ai understood my work based on my statements"*

> *"The lab was super fun, especially PrairieLearn as it was a completely new experience"*

Looking beyond the novelty of the activities themselves, students also reported that the activites were engaging and enjoyable with the terms "fun" and "enjoyable" being used frequently in the responses.

> *"I just want to say I loved the PrairieLearn tasks"*

> *"The PraireLearn part was also very fun to do and use"*

These sentiments indicate that the activities were enjoyable, engaging, and were valued by students as a part of their learning experience in the age of LLM code generation tools. These sentiments are particularly powerful when considering the potential for these activities to be used in formative activities.

4.3.2 Enhanced Comprehension of Code: Many students reported that the exercises helped develop their understanding of code and their ability to communicate a code's purpose (Figure 6). This sentiment was reflected in short answer responses as well with students often reporting that the activity was helpful in developing their ability to read and comprehend code.

> *"I think it is very useful in really comprehending chunks of code in terms of their purpose rather than its individual tasks."*

> *"It actually gets people to read and understand code, and try to figure out what the original code was supposed to do. I feel that my ability to describe the actual action not just the steps going on improved as I read the code more"*

"PrairieLearn [activities] gave me much better insight into the connection between natural language and coding, because small changes in wording had a large impact on the code."

4.3.3 Concerns Relating to Overreliance and LLM Capabilities: Despite the overwhelmingly positive response to the activities, students were dubious about the ability of AI to supplant traditional coding activities particularly in the context of more complex tasks.

"Pretty cool. The code was simplistic enough for an AI to generate a valid result - but I don't think this would work in more complex problems."

"The task was fairly helpful in understanding what code means. However, I feel like it only has limited use in helping me to write code more easily and efficiently."

From previous sections, it is clear that students generally see the activities as valuable, particularly for building code comprehension skills. However, responses from this section reveal some wariness about relying too heavily on AI-generated code for learning and assessment, suggesting that while the task is seen as an interesting complement to traditional methods, a balance is needed. One student expressed this explicitly: *"So, I think I mixture of the AI model question and regular coding question will be good, but more regular coding question than the other one".*

4.4 RQ4: Students' use of feedback

Though some students reported getting all exercises correct on their first attempt, the majority reported needing to modify at least some of their prompts. Three overarching themes emerged from analysis of students' comments on their debugging process: revisiting original code, comparing generated code to given code, and evaluating test cases. Each of these sheds light on how students utilized the feedback provided by the system and, in the event they did not, the alternative approaches they took to achieve success.

4.4.1 Revisiting original code: For those students who did not report using the feedback provided by the system, the majority stated they went about modifying their prompts by simply revisiting the original code and attempting to spot errors or use more specific language.

"1) I reread the [given] code and compared it to text I wrote and tried to find any differences. 2) I did not really use the feedback system."

"I reread the code over and over again, scratched my head, and had a longgg think. I didn't look at the given feedback, probably should have."

In several instances students indicated that revisiting the original code took the form of tracing the code with example inputs in order to determine its behavior and modify their prompt:

"When scoring incorrect I would just run the code in my head (or book) with different values and compare and see how the initial inputs were altered to produce the output."

"If my answer was incorrect, I tried solving it by hand (with example values) to get a better understanding my code works - it helped me get the right answer."

In these cases, though the feedback was not utilized, their approaches to debugging their prompts were still productive from the perspective of developing code comprehension.

4.4.2 Comparing Generated Code to Given Code and Evaluating Test Cases. The majority of students reported using some combination of comparing the generated code to the given code and evaluating the results of test cases when debugging their prompts.

"I'd use the system's feedback, especially the generated code, to understand where the misunderstandings were and adjust my prompts accordingly. Test cases would also guide any refinements."

"There was only one instance in which my initial response was incorrect. When I came across this issue, I compared my code to the source code we were required to try and achieve and found my error. After finding the error, I changed some of the wording of my prompt and clarified it to be easier to interpret by the AI. By using the code generated, and reviewing the test cases I was able to make the required changes necessary to complete the question and achieve 100%."

In particular, several students noted how small ambiguities in their language could lead to plausible but incorrect code being generated by the LLM.

"I first looked at the way I worded my submission and checked to see whether edge cases would appear. For instance, "if (i > 0)" would be if the value i is "greater than zero" but I had submitted "non-zero", meaning negative numbers would be included. I then try to debug the code the AI had written and see what had changed within the logic of the newly written code and improve my wording."

"In the task where we had to prompt to create a function that returns the last index of zero, I passed all tests initially apart from the array with multiple zeros. I didn't fully understand the code on my first attempt, so I compared the generated code with the code we needed to output, which helped me realised was the LAST index we needed to prompt for."

Others indicated how the test cases in particular were useful in helping them account for edge cases and modify their prompt to cover them.

"I read the test cases, and for one of them, I didn't specify that it must sum the values greater than 0. Once I saw the test cases I saw that some values were less than 0 so I changed that in the prompt."

"I would read through the code and check what the ai generator may be misinterpreting then tried again. for question 3 [find index of last zero] I had to check what the inputs where and see if it was compatible with the function example 0,0,0,0, The consecutive zeros was not anticipated."

4.5 RQ5: Student perceptions of LLM capabilities

Three primary themes emerged from the thematic analysis of student responses around the capability of LLMs for generating code.

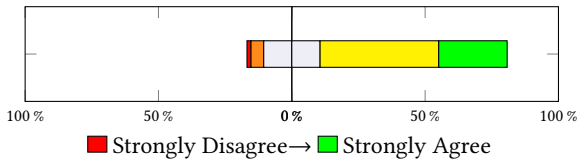


Figure 6: "Students should be taught how to use Generative AI tools effectively for their future careers."

These captured the perceived potential for LLMs to transform coding practices as well as concerns about their limitations and implications.

4.5.1 Efficiency and Time-Saving. A key theme was the efficiency and time-saving potential of LLMs in generating code. Many students appreciated the ability of LLMs to automate coding tasks, highlighting their potential for speeding up development processes. For instance, one response captured this sentiment:

> *"To me, the current situation and ability of AI has already proven very useful as it help save time on simple basic but tedious function to code out."*

Another student echoed this positive view, while also noting some reservations:

> *"While it is efficient for machines to program for us, the hands-on proper learning would be lost. However, from a business perspective, large language models that generate code may reduce expenditures by replacing human programmers."*

Some of the individual responses relating to this theme highlighted that the efficiency gains would be useful for professional developers, but they were less optimistic regarding use by novices.

> *"I think using large language models to generate code would be a great idea in terms of use by experienced/ expert programmers, as it can save time. However, I believe using such large language models could be detrimental for newer/inexperienced programmers who may just use the large language model to be lazy and delay learning essential skills that would otherwise benefit them in the long run."*

4.5.2 Impressive capabilities. Many responses indicated that students were impressed at the effectiveness of the models, sometimes to the point of amazement, for generating code.

> *"The capability of large language models really surprised me, as it interpreted my pseudocode for it to write perfectly, despite my subpar English writing skills. I think the capability of large language models is amazing, however, scary as well."*

> *"Its amazing how good AI models are at generating code which works well and is very efficient."*

> *"I was also amazed at how accurate it was at making the code even when you leave some small details out it was able to assume and add those parts."*

4.5.3 Concerns about Accuracy and Reliability. Not all students were impressed however, and concerns regarding the accuracy and reliability of LLM-generated code were also prominent. Students questioned the models' ability to handle complex or nuanced tasks and recognized the potential need for human oversight. One student expressed this concern by stating:

> *"There is a problem with amateur coders who might not fully understand the code generated which could lead to code generated that is different from the intended purpose is supposed to be."*

Another student highlighted a similar issue:

> *"These large language models can really help out when you get stuck. However, the code it produces is not always correct, so you still need to test and debug it, just in case."*

In summary, there was a general perception from students that LLMs can be transformative tools in the programming landscape, capable of improving efficiency, but with concerns over reliability and the impact on novice programmers' learning outcomes. These findings highlight the benefits of learning to leverage LLMs through tasks such as EiPE questions, as well as a number of important aspects of their use (e.g., the importance of code comprehension, and validating generated code).

5 DISCUSSION

As takeaways from this work, we wish to highlight three key outcomes. First, we have shown that students are generally successful and have positive opinions of the code explanation activity (Section 5.1). Second, our results indicate that there appears to be a natural synergy between successfully prompting an LLM to generate code and many of the goals of traditional code comprehension exercises such as EiPE questions (Section 5.2). Finally, though students were largely positive regarding the capabilities of LLMs for the purposes of code generation, they also expressed wariness regarding their potential impacts (Section 5.3).

5.1 Students' Success and Enjoyment of the Exercises

When adopting a new exercise into a course or curriculum it is often important to consider outcomes such as the enjoyment of the exercise by students and the degree to which they find the exercises engaging. Overall, students were typically able to complete all tasks, across both labs, in under three attempts and all tasks had an overall correctness rate of over 95%. In particular, students widely reported enjoying the novelty of the exercises and appreciating the use of AI supported technology being integrated into their course. This supports their use in introductory courses as generating a viable prompt appears to be easily achievable for most students, in addition to being engaging. This in particular supports their usage in formative assessments, where the goal is to provide students with engaging exercises that support their learning.

5.2 The Intersection of Code Comprehension and Prompting for Code Generation

Given the goal of these exercises is to both facilitate the development of code comprehension skills and expose students to the process of prompting LLMs, it is important to consider success in both contexts. Though our findings suggest that *relational* prompts are generally more effective than *multistructural* for producing correct code, it is worth noting that it is possible to construct a *multistructural* response that consistently generates correct code.

This distinguishes this grading process from other approaches to grading EiPE questions which place a heavy emphasis on students providing a *relational* description of the code [6, 36, 38]. Future work should investigate methods of guiding students towards *relational* answers while maintaining the scalability of the current system.

Looking beyond the grading process of the exercises, it is also important to consider the ways in which the feedback provided by this grading approach supports the goals of code comprehension exercises. Many students reported that they were able to use the feedback to gain insights into the relationship between the words used in their prompt and the code it generated. Similarly, students reported the test cases caused them to reflect on edge cases they had previously not considered and provide prompts which more precisely defined the function's behavior. These findings coupled with students' overwhelming agreement that the activity both caused them to engage with reading and understanding the provided code and that this exercise was an effective means of assessing their comprehension abilities suggests this activity effectively addresses the shortcomings relating to feedback and transparency of prior ASAG systems for EiPE questions [1, 19, 26, 34].

5.3 Students' Perceptions and Concerns of LLM Code Generation Capabilities

Students generally reported being impressed by the capabilities of LLMs for code generation and valued them as a means of improving efficiency and productivity. However, these positive sentiments were tempered by a variety of concerns which should be considered when integrating this approach, and LLM supported interventions more generally, into a course. The most common concerns surrounded novices becoming overly dependent on LLMs for code generation thus missing the development of this critical skill, and the potential for LLMs to generate poor quality and incorrect code. These sentiments are consistent with prior work on novices' perceptions of LLMs for code generation [43] and highlight an important consideration for instructors who may wish to integrate this and other approaches for teaching students to leverage LLMs into their curriculum. Instructors may wish to address these concerns outright when such exercises are introduced. This may be done by indicating that with the development of skills such as prompting, other skills must be developed such as testing and validating the code generated by an LLM. It might also be useful to highlight that LLMs do not replace knowledge of the basics, as in order to effectively use LLMs for code generation, students must have a strong understanding of the code they are attempting to generate in order to successfully evaluate it.

5.4 Explain in Plain Language Questions

Fundamental to EiPE questions is the final 'E': English. As noted by others [32], this serves as a barrier to adoption in non-English speaking contexts and for English as a Second Language (ESL) students. In our analysis of students' prompts, we observed that in several instances students were able to successfully generate code from prompts that were written in Chinese or contained a mix of English and Chinese words. This enables this autograding mechanism to support "Explain in Plain Language" (EiPL) questions. We would expect EiPL questions to work in a wide variety of languages given the impressive capabilities of modern LLMs for language translation, however it may perform better with respect to languages that are better represented in the training data. Nevertheless, the ability to author a single question which natively supports code comprehension in a variety of languages represents a significant advancement on traditional EiPE questions. We see great potential in EiPL questions for improving access to computing education, and this warrants future research.

该函数foo 的目的是检查字符串 str1 是否包含字符串 str2 作为子字符串。它首先获取两个字符串的长度，然后通过两层嵌套循环来检查 str1 中的每个子字符串是否与 str2 匹配。如果找到匹配的子字符串，函数返回 1；如果遍历完 str1 后仍未找到匹配的子字符串，函数返回 0。	The purpose of the function foo is to check whether the string str1 contains the string str2 as a substring. It first gets the lengths of the two strings and then goes through two levels of nested loops to check whether each substring in str1 matches str2. If a matching substring is found, the function returns 1; if no matching substring is found after traversing str1, the function returns 0.

Figure 7: A code explanation in Chinese submitted by a student, which successfully solved the task, with English translation shown on the right.

6 LIMITATIONS

We acknowledge several limitations of this study. The range of difficulty of the questions used in the study was relatively narrow, with the tasks being on the easier side. Future work should examine a broader spectrum of difficulty levels. In addition, our evaluation focused on the immediate outcomes related to the success and perceptions of the autograding system and its impact on comprehension skills. Long-term retention of comprehension abilities and the ability for students to generalize their skills beyond the context of the study would be important to explore in future research. Finally, the study was conducted relatively late in the course, during weeks 8 and 10, meaning students already had some experience with programming. Future studies should aim to assess the impact of such systems on novices at the very start of their programming education.

7 CONCLUSION

The ability to read and comprehend code has always been a core skill for programming, and is likely to be of particular importance as we enter a world with LLMs. To that end, we investigated a form of code prompting question, in the tradition of EiPE, which aims to teach and assess code comprehension in a way that is scalable and readily beneficial to preparing students for working with LLMs. We found students were remarkably successful at completing code explanation prompts and that these prompts reward the types of answers instructors desire the most: that is, *relational* answers.

Students' reflections on these questions were also heartening. Students found the feedback for these prompts helpful and often used the feedback to improve their answers. Additionally, students showed a healthy skepticism for LLMs in general, appreciating their power but being aware that they cannot replace code literacy, and programming skills broadly, as a core competency for computing professionals. Our results show that not only does LLM prompting serve as a useful exercise in its own right, but it has the potential to be a core part of helping students navigate using LLM-powered tools in the future.

REFERENCES

[1] Sushmita Azad. 2020. *Lessons learnt developing and deploying grading mechanisms for EiPE code-reading questions in CS1 classes.* Ph. D. Dissertation.

[2] Sushmita Azad, Binglin Chen, Maxwell Fowler, Matthew West, and Craig Zilles. 2020. Strategies for deploying unreliable AI graders in high-transparency high-stakes exams. In *Artificial Intelligence in Education: 21st International Conference, AIED 2020, Ifrane, Morocco, July 6–10, 2020, Proceedings, Part I 21*. Springer, 16–28. https://doi.org/10.1007/978-3-030-52237-7_2

[3] John B Biggs and Kevin F Collis. 2014. *Evaluating the quality of learning: The SOLO taxonomy (Structure of the Observed Learning Outcome)*. Academic Press.

[4] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative research in psychology* 3, 2 (2006), 77–101.

[5] Steven Burrows, Iryna Gurevych, and Benno Stein. 2015. The eras and trends of automatic short answer grading. *International journal of artificial intelligence in education* 25 (2015), 60–117. https://doi.org/10.1007/s40593-014-0026-8

[6] Binglin Chen, Sushmita Azad, Rajarshi Haldar, Matthew West, and Craig Zilles. 2020. A validated scoring rubric for explain-in-plain-english questions. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education*. 563–569. https://doi.org/10.1145/3328778.3366879

[7] Bruno Pereira Cipriano and Pedro Alves. 2023. Gpt-3 vs object oriented programming assignments: An experience report. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*. 61–67. https://doi.org/10.1145/3587102.3588814

[8] Bruno Pereira Cipriano, Pedro Alves, and Paul Denny. 2024. A Picture Is Worth a Thousand Words: Exploring Diagram and Video-Based OOP Exercises to Counter LLM Over-Reliance. arXiv:2403.08396 [cs.SE]

[9] J.H.I.I. Cross, T.D. Hendrix, and L.A. Barowski. 2002. Using the debugger as an integral part of teaching CS1. In *32nd Annual Frontiers in Education*, Vol. 2. F1G–F1G. https://doi.org/10.1109/FIE.2002.1158137

[10] Paul Denny, Brett A Becker, Juho Leinonen, and James Prather. 2023. Chat Overflow: Artificially Intelligent Models for Computing Education-renAIssance or apocAIypse?. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*. 3–4. https://doi.org/10.1145/3587102.3588773

[11] Paul Denny, David H. Smith IV, Max Fowler, James Prather, Brett A. Becker, and Juho Leinonen. 2024. Explaining Code with a Purpose: An Integrated Approach for Developing Code Comprehension and Prompting Skills. In *Proceedings of the 2024 Conference on Innovation and Technology in Computer Science Education V. 1* (Milan, Italy) *(ITiCSE 2024)*. Association for Computing Machinery, New York, NY, USA, 7 pages. https://doi.org/10.1145/3649217.3653587

[12] Paul Denny, Viraj Kumar, and Nasser Giacaman. 2023. Conversing with Copilot: Exploring prompt engineering for solving CS1 problems using natural language. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*. 1136–1142. https://doi.org/10.1145/3545945.3569823

[13] Paul Denny, Juho Leinonen, James Prather, Andrew Luxton-Reilly, Thezyrie Amarouche, Brett A. Becker, and Brent N. Reeves. 2024. Prompt Problems: A New Programming Exercise for the Generative AI Era. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (Portland, USA) *(SIGCSE 2024)*. ACM, New York, NY, USA, 296–302. https://doi.org/10.1145/3626252.3630909

[14] Paul Denny, Andrew Luxton-Reilly, Ewan Tempero, and Jacob Hendrickx. 2011. CodeWrite: Supporting Student-Driven Practice of Java. In *Proceedings of the 42nd ACM Technical Symposium on Computer Science Education* (Dallas, TX, USA) *(SIGCSE '11)*. Association for Computing Machinery, New York, NY, USA, 471–476. https://doi.org/10.1145/1953163.1953299

[15] Paul Denny, James Prather, Brett A. Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N. Reeves, Eddie Antonio Santos, and Sami Sarsa. 2024. Computing Education in the Era of Generative AI. *Commun. ACM* 67, 2 (Jan 2024), 56–67. https://doi.org/10.1145/3624720

[16] John Edwards, Joseph Ditton, Dragan Trninic, Hillary Swanson, Shelsey Sullivan, and Chad Mano. 2020. Syntax exercises in CS1. In *Proceedings of the 2020 ACM Conference on International Computing Education Research*. 216–226. https://doi.org/10.1145/3372782.3406259

[17] Nigel Fernandez, Aritra Ghosh, Naiming Liu, Zichao Wang, Benoît Choffin, Richard Baraniuk, and Andrew Lan. 2022. Automated scoring for reading comprehension via in-context bert tuning. In *International Conference on Artificial Intelligence in Education*. Springer, 691–697. https://doi.org/10.1007/978-3-031-11644-5_69

[18] James Finnie-Ansley, Paul Denny, Brett A Becker, Andrew Luxton-Reilly, and James Prather. 2022. The robots are coming: Exploring the implications of openai codex on introductory programming. In *Proceedings of the 24th Australasian Computing Education Conference*. 10–19. https://doi.org/10.1145/3511861.3511863

[19] Max Fowler, Binglin Chen, Sushmita Azad, Matthew West, and Craig Zilles. 2021. Autograding" Explain in Plain English" questions using NLP. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*. 1163–1169. https://doi.org/10.1145/3408877.3432539

[20] Max Fowler, Binglin Chen, and Craig Zilles. 2021. How should we 'Explain in plain English'? Voices from the Community. In *Proceedings of the 17th ACM conference on international computing education research*. 69–80. https://doi.org/10.1145/3446871.3469738

[21] Max Fowler, David H Smith IV, Mohammed Hassan, Seth Poulsen, Matthew West, and Craig Zilles. 2022. Reevaluating the relationship between explaining, tracing, and writing skills in CS1 in a replication study. *Computer Science Education* 32, 3 (2022), 355–383. https://doi.org/10.1080/08993408.2022.2079866

[22] Lucas Busatta Galhardi and Jacques Duílio Brancher. 2018. Machine learning approach for automatic short answer grading: A systematic review. In *Advances in Artificial Intelligence-IBERAMIA 2018: 16th Ibero-American Conference on AI, Trujillo, Peru, November 13-16, 2018, Proceedings 16*. Springer, 380–391. https://doi.org/10.1007/978-3-030-03928-8_31

[23] Qiang Hao, David H Smith IV, Lu Ding, Amy Ko, Camille Ottaway, Jack Wilson, Kai H Arakawa, Alistair Turcan, Timothy Poehlman, and Tyler Greer. 2022. Towards understanding the effective design of automated formative feedback for programming assignments. *Computer Science Education* 32, 1 (2022), 105–127. https://doi.org/10.1080/08993408.2020.1860408

[24] Qiang Hao, Jack P Wilson, Camille Ottaway, Naitra Iriumi, Kai Arakawa, and David H Smith. 2019. Investigating the Essential of Meaningful Automated Formative Feedback for Programming Assignments. In *2019 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 151–155. https://doi.org/10.1109/VLHCC.2019.8818922

[25] Owen Henkel, Libby Hills, Bill Roberts, and Joshua McGrane. 2023. Can LLMs Grade Short-answer Reading Comprehension Questions: Foundational Literacy Assessment in LMICs. *arXiv preprint arXiv:2310.18373* (2023).

[26] Silas Hsu, Tiffany Wenting Li, Zhilin Zhang, Max Fowler, Craig Zilles, and Karrie Karahalios. 2021. Attitudes surrounding an imperfect AI autograder. In *Proceedings of the 2021 CHI conference on human factors in computing systems*. 1–15. https://doi.org/10.1145/3411764.3445424

[27] Jacqueline Hundley. 2008. A review of using design patterns in CS1. In *Proceedings of the 46th Annual Southeast Regional Conference on XX*. 30–33. https://doi.org/10.1145/1593105.1593113

[28] Cruz Izu and Claudio Mirolo. 2023. Exploring CS1 Student's Notions of Code Quality. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*. 12–18. https://doi.org/10.1145/3587102.3588808

[29] Majeed Kazemitabaar, Xinying Hou, Austin Henley, Barbara J Ericson, David Weintrop, and Tovi Grossman. 2023. How novices use LLM-based code generators to solve CS1 coding tasks in a self-paced learning environment. *arXiv preprint arXiv:2309.14049* (2023).

[30] Majeed Kazemitabaar, Runlong Ye, Xiaoning Wang, Austin Z. Henley, Paul Denny, Michelle Craig, and Tovi Grossman. 2024. CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, Hawaii, USA) *(CHI '24)*. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3613904.3642773

[31] Natalie Kiesler and Daniel Schiffner. 2023. Large Language Models in Introductory Programming Education: ChatGPT's Performance and Implications for Assessments. *arXiv preprint arXiv:2308.08572* (2023).

[32] Viraj Kumar. 2021. Refute: An Alternative to 'Explain in Plain English' Questions. In *Proceedings of the 17th ACM Conference on International Computing Education Research*. 438–440. https://doi.org/10.1145/3446871.3469791

[33] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).

[34] Tiffany Wenting Li, Silas Hsu, Max Fowler, Zhilin Zhang, Craig Zilles, and Karrie Karahalios. 2023. Am I Wrong, or Is the Autograder Wrong? Effects of AI Grading Mistakes on Learning. In *Proceedings of the 2023 ACM Conference on International Computing Education Research-Volume 1*. 159–176. https://doi.org/10.1145/3568813.3600124

[35] Raymond Lister, Colin Fidge, and Donna Teague. 2009. Further evidence of a relationship between explaining, tracing and writing skills in introductory programming. *Acm sigcse bulletin* 41, 3 (2009), 161–165. https://doi.org/10.1145/1595496.1562930

[36] Raymond Lister, Beth Simon, Errol Thompson, Jacqueline L Whalley, and Christine Prasad. 2006. Not seeing the forest for the trees: novice programmers and the SOLO taxonomy. *ACM SIGCSE Bulletin* 38, 3 (2006), 118–122. https://doi.org/10.1145/1140123.1140157

[37] Mike Lopez, Jacqueline Whalley, Phil Robbins, and Raymond Lister. 2008. Relationships between reading, tracing and writing skills in introductory programming. In *Proceedings of the fourth international workshop on computing education research*. 101–112. https://doi.org/10.1145/1404520.1404531

[38] Laurie Murphy, Renée McCauley, and Sue Fitzgerald. 2012. 'Explain in plain English'questions: implications for teaching. In *Proceedings of the 43rd ACM technical symposium on Computer Science Education*. 385–390. https://doi.org/10.1145/2157136.2157249

[39] Greg L Nelson, Benjamin Xie, and Amy J Ko. 2017. Comprehension first: evaluating a novel pedagogy and tutoring system for program tracing in CS1. In *Proceedings of the 2017 ACM conference on international computing education research*. 2–11. https://doi.org/10.1145/3105726.3106178

[40] Priti Oli, Rabin Banjade, Jeevan Chapagain, and Vasile Rus. 2023. Automated Assessment of Students' Code Comprehension using LLMs. *arXiv preprint arXiv:2401.05399* (2023).

[41] Linus Östlund, Niklas Wicklund, and Richard Glassey. 2023. It's Never too Early to Learn About Code Quality: A Longitudinal Study of Code Quality in First-year Computer Science Students. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*. 792–798. https://doi.org/10.1145/3545945.3569829

[42] James Prather, Paul Denny, Juho Leinonen, Brett A Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, et al. 2023. The robots are here: Navigating the generative ai revolution in computing education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*. 108–159. https://doi.org/10.1145/3623762.3633499

[43] James Prather, Paul Denny, Juho Leinonen, David H Smith IV, Brent N Reeves, Stephen MacNeil, Brett A Becker, Andrew Luxton-Reilly, Thezyrie Amarouche, and Bailey Kimmel. 2024. Interactions with Prompt Problems: A New Way to Teach Programming with Large Language Models. *arXiv preprint arXiv:2401.10759* (2024).

[44] Brent Reeves, Sami Sarsa, James Prather, Paul Denny, Brett A Becker, Arto Hellas, Bailey Kimmel, Garrett Powell, and Juho Leinonen. 2023. Evaluating the performance of code generation models for solving Parsons problems with small prompt variations. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*. 299–305. https://doi.org/10.1145/3587102.3588805

[45] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[46] David H Smith IV and Craig Zilles. 2023. Code Generation Based Grading: Evaluating an Auto-grading Mechanism for" Explain-in-Plain-English" Questions. *arXiv preprint arXiv:2311.14903* (2023).

[47] David H Smith IV and Craig Zilles. 2024. Evaluating Large Language Model Code Generation as an Autograding Mechanism for" Explain in Plain English" Questions. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 2*. 1824–1825. https://doi.org/10.1145/3626253.3635542

[48] Allison Elliott Tew and Mark Guzdial. 2010. Developing a validated assessment of fundamental CS1 concepts. In *Proceedings of the 41st ACM technical symposium on Computer science education*. 97–101. https://doi.org/10.1145/1734263.1734297

[49] Matthew West, Geoffrey L Herman, and Craig Zilles. 2015. PrairieLearn: Mastery-based online problem solving with adaptive scoring and recommendations driven by machine learning. In *2015 ASEE Annual Conference & Exposition*. 26–1238.

[50] Benjamin Xie, Dastyni Loksa, Greg L Nelson, Matthew J Davidson, Dongsheng Dong, Harrison Kwik, Alex Hui Tan, Leanne Hwa, Min Li, and Amy J Ko. 2019. A theory of instruction for introductory programming skills. *Computer Science Education* 29, 2-3 (2019), 205–253. https://doi.org/10.1080/08993408.2019.1565235